DevOps-IT Home Assignment

Item	Details
Assignment type	Backend/API home assignment
Primary focus	Read-only GitLab reporting using the GitLab REST API
Required deliverable	Web service packaged in a Docker container
Bonus deliverable	MCP server exposing the same reporting functions

Goal

Implement a small service that integrates with the GitLab REST API and provides yearly reports for:

- Issues created in a specific year
- Merge requests created in a specific year

The implementation should support both of the following scopes:

- A single GitLab project/repository
- The entire GitLab instance, according to the permissions of the provided token

The assignment is intentionally read-only. The application must not create, update, or delete GitLab resources.

Technical Requirements

GitLab Integration

Use GitLab REST API v4. The application should receive the GitLab URL and token through environment variables:

```
GITLAB_URL=https://gitlab.example.com
GITLAB_TOKEN=<personal-access-token-or-project-token>
```

The token must have permissions to read the relevant projects, issues, and merge requests.

Task 1 - Get Issues by Year

Implement a function that returns GitLab issues created during a given year for specific repo, return all Gitlab issues if project id not provided.

```
get_issues_by_year(year, project_id_or_path=None)
```

Parameter	Required	Description
year	Yes	4-digit year, for example 2025
project_id_or_path	No	GitLab project ID or URL-encoded path. If not provided, return results from the entire GitLab instance according to token permissions.

Task 2 - Get Merge Requests by Year

Implement a function that returns GitLab merge requests created during a given year, return all Gitlab MRs if project id not provided.

```
get_merge_requests_by_year(year, project_id_or_path=None)
```

Parameter	Required	Description
year	Yes	4-digit year, for example 2025
project_id_or_path	No	GitLab project ID or URL-encoded path. If not provided, return results from the entire GitLab instance according to token permissions.

Task 3 - Expose as a Web Service

Expose the functions through a small HTTP API. You may use any language or framework, for example Python with FastAPI or Flask, Go, or Node.js. Bash is allowed only if the service behavior is clean and maintainable.

Required API endpoints

Endpoint	Description
GET /health	Returns {"status": "ok"}
GET /issues?year=2025	Returns issues from the entire GitLab instance.
GET /issues?year=2025&project=my-group%2Fmy-project/ID	Returns issues from a specific project.
GET /merge-requests?year=2025	Returns merge requests from the entire GitLab instance.
GET /merge-requests?year=2025&project=my-group%2Fmy-project/ID	Returns merge requests from a specific project.

Error Handling Requirements

Scenario	Expected status
Missing year	400 Bad Request
Invalid year format	400 Bad Request
GitLab token missing	500 Internal Server Error or clear startup failure
GitLab project not found	404 Not Found
GitLab authentication failed	401 Unauthorized
GitLab permission denied	403 Forbidden

Task 4 - Dockerize the Service

Create a Dockerfile that builds and runs the web service. The container should be configurable using environment variables.

Example (can implement as you wish)

```
docker run --rm -p 8080:8080 \
  -e GITLAB_URL="https://gitlab.example.com" \
  -e GITLAB_TOKEN="glpat-xxxx" \
  gitlab-yearly-report-service
```

The service should be available on:

```
http://localhost:8080
```

Bonus Task - MCP Integration

Implement an MCP server that exposes the same functionality as tools.

Required MCP tools

```
get_issues_by_year
```

```
get_merge_requests_by_year
```

Deliverables

Please provide one of the following:

- A compressed archive containing the project files
- A link to a public Git repository

The submission should include:

1. `README.md`
   `Dockerfile`
   `Application source code`
   `Example configuration` (can be part of readme)
   `Example curl commands` (can be part of readme)
   Bonus: how to run and test the MCP server, if implemented

Optional Local GitLab Playground

Candidates may test against GitLab.com or a locally running GitLab instance. For a local playground, use a GitLab Docker container that matches GitLab 18.10 or newer. The exact image tag should be selected according to the latest available GitLab 18.10+ release at the time of testing.

```
GITLAB_HOME=/tmp/gitlab
sudo docker run --detach \
  --hostname gitlab.example.com \
  --env GITLAB_OMNIBUS_CONFIG="external_url 'http://gitlab.example.com'" \
  --publish 443:443 --publish 80:80 --publish 22:22 \
  --name gitlab \
  --restart always \
  --volume $GITLAB_HOME/config:/etc/gitlab \
  --volume $GITLAB_HOME/logs:/var/log/gitlab \
  --volume $GITLAB_HOME/data:/var/opt/gitlab \
  --shm-size 256m \
  gitlab/gitlab-ee: 18.10.5-ee.0
```

```
After GitLab starts, create groups, projects, issues, and merge requests for testing. Then create a token with
the required read permissions and use it with the service.
```